

Document 02 — TRD

Technical Requirements Document for NowWise

Product Name

NowWise

Internal Platform Name

NowWise OS

Module Covered

Telegram-first, voice-first AI lifestyle operating system for food logging, product lookup, grocery decisions, restaurant guidance, habit reminders, weekly reports, product intelligence, and admin review.

1. Technical Summary

NowWise is a **Telegram-first and voice-first AI lifestyle operating system**.

The user sends:

- Voice note
- Text message
- Product name
- Food photo
- Grocery receipt
- Restaurant menu
- Reminder request

The system should:

1. Receive the message from Telegram.
2. Capture real-time timestamp and timezone automatically.
3. Detect input type.
4. Transcribe voice if needed.
5. Classify intent.
6. Extract food/product/habit entities.
7. Store the event in a life timeline.
8. Use product, food, habit, and behavior intelligence.

9. Reply with personalized guidance.
10. Create reminders if needed.
11. Generate weekly behavior reports.
12. Route low-confidence cases to admin review.

Core technical principle:

Voice and text must work fast. Photo should never block the user.

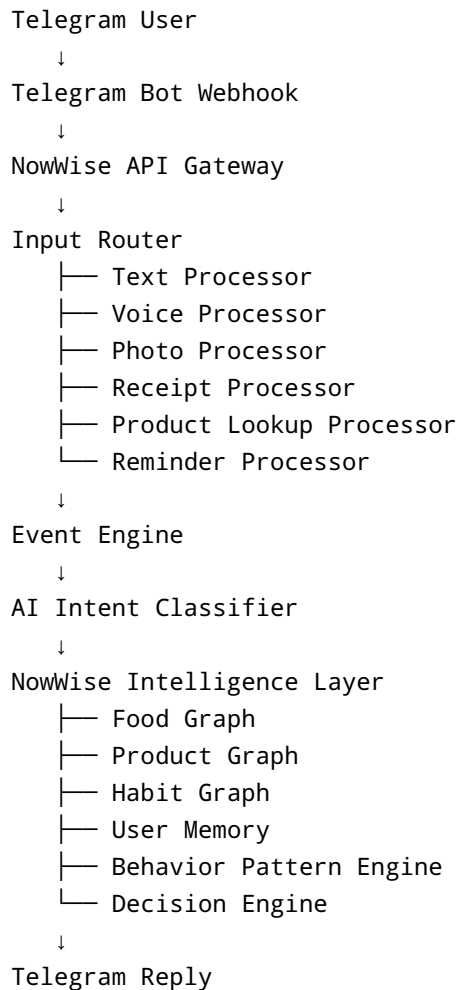
2. Final Stack Decision

Layer	Selected Technology	Reason
Frontend	Next.js with TypeScript	Strong full-stack web framework, good for admin dashboard and future user portal
Styling	Tailwind CSS	Fast UI building, consistent design
UI Components	shadcn/ui + Radix UI	Clean dashboard components
Backend API	Next.js API routes + server actions where appropriate	Keeps web/admin/backend in one repo
Background Workers	Python FastAPI workers + Node workers where needed	Python is better for audio/AI/FFmpeg jobs; Node fits Telegram and app logic
Database	Supabase Postgres	Structured relational data, auth, storage, realtime possibilities
Auth	Supabase Auth for admin dashboard	Telegram users do not need email login in MVP
File Storage	Supabase Storage	Temporary audio/photo/receipt storage
Queue	Redis + BullMQ initially OR Supabase queue if keeping stack simple	Required for voice/photo/background processing
Hosting	Vercel for frontend/API, Railway/Fly.io/Render for workers, Supabase for DB/storage	Vercel alone is not enough for long-running audio jobs
Telegram Bot	Telegram Bot API webhook	Primary user interface
Voice Engine	OpenWhisper-inspired local ASR service + cloud fallback	Privacy-first, cost-control, no Wispr vendor lock-in
Audio Conversion	FFmpeg	Converts Telegram OGG/OPUS to WAV

Layer	Selected Technology	Reason
AI Reasoning	OpenAI / Claude / Gemini provider abstraction	Do not hard-code only one model
Vector Search	pgvector in Supabase or Qdrant later	For product/food/user memory search
Monitoring	Sentry + structured logs	Needed for voice failures, AI errors, worker issues
Payments	Stripe later	Not needed in MVP
Email	Resend later	For admin reports, not core MVP

3. Architecture Decision

3.1 High-Level Architecture



3.2 Voice Architecture

```
Telegram Voice Note
↓
Audio Fetcher
↓
Temporary Audio Storage
↓
FFmpeg Conversion
↓
Voice Provider Router
├─ Local OpenWhisper-style ASR
├─ Whisper.cpp
├─ Parakeet / sherpa-onnx
├─ OpenAI/Groq/Deepgram fallback
└─ Wispr Flow optional fallback
↓
Transcript
↓
Transcript Cleaner
↓
Intent Classifier
↓
Food/Product/Reminder Action
```

Important rule:

Do not embed the full OpenWhisper Electron desktop app into NowWise production backend.

Use OpenWhisper for:

- Local ASR reference
- Self-hosted transcription pattern
- Whisper/Parakeet ideas
- Provider routing inspiration
- Internal founder/team dictation

Do not use OpenWhisper for:

- Full backend
- User dashboard

- Telegram bot
 - NowWise intelligence layer
 - Reminder system
 - Product database
-

4. Frontend

4.1 Framework

Next.js with TypeScript

Use for:

- Admin dashboard
 - Internal product compliance portal
 - User dashboard later
 - Privacy center
 - Weekly report preview
 - Product database correction
 - AI review queue
-

4.2 Styling

Tailwind CSS

Reason:

- Fast development
 - Good dashboard layout
 - Easy responsive design
 - Works well with shadcn/ui
-

4.3 UI Library

Use:

- shadcn/ui
- Radix UI
- Lucide Icons
- React Hook Form
- Zod validation

4.4 Frontend Pages

```
/app
  /admin
    /dashboard
    /users
    /events
    /voice-events
    /products
    /product-reviews
    /reminders
    /weekly-reports
    /privacy-requests
    /settings

  /user
    /dashboard
    /timeline
    /reminders
    /weekly-story
    /privacy

  /api
    /telegram
    /voice
    /events
    /products
    /reminders
    /reports
```

5. Backend

5.1 Backend Runtime

Use a hybrid backend:

Backend Part	Technology
Telegram webhook	Next.js API route or Node service
Core app APIs	Next.js API routes

Backend Part	Technology
Audio processing	Python worker
ASR service	Python FastAPI service
Background jobs	BullMQ workers / Python workers
AI reasoning	Provider abstraction
Admin API	Next.js API routes

5.2 Why Hybrid Backend?

Node/Next.js is good for:

- Telegram webhook
- Admin APIs
- Product database APIs
- Dashboard
- Supabase integration
- Authentication

Python is better for:

- FFmpeg processing
 - Local ASR service
 - Whisper/Parakeet integration
 - Audio pipeline
 - Batch processing
 - AI evaluation jobs
-

6. Database

6.1 Provider

Supabase Postgres

Reason:

- Strong relational structure
- Built-in Auth
- Storage
- Row-level security
- Good for admin dashboards

- Easy to use with Next.js

6.2 Required Extensions

Use if needed:

```
pgcrypto
uuid-oss
pgvector
pg_trgm
```

Purpose:

Extension	Purpose
pgcrypto	UUIDs/security
uuid-oss	UUID generation
pgvector	Embedding search
pg_trgm	Fuzzy product search

7. Core Database Tables

7.1 users

Stores Telegram users.

Column	Type	Notes
id	uuid	Primary key
telegram_user_id	text	Unique
telegram_chat_id	text	For replies
name	text	Optional
username	text	Telegram username
timezone	text	Example: Asia/Kolkata
language_preference	text	en, hi, te, hinglish

Column	Type	Notes
created_at	timestampz	Auto
updated_at	timestampz	Auto

7.2 user_profiles

Stores user goals and preferences.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
primary_goal	text	digestion, sugar_control, weight_control, energy
diet_type	text	veg, non_veg, egg, vegan
allergies	jsonb	Optional
health_notes	text	User-entered, not diagnosis
privacy_mode	text	normal/private/local_preferred
created_at	timestampz	Auto
updated_at	timestampz	Auto

7.3 user_events

Universal timeline table.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
event_type	text	food_log, product_lookup, reminder, grocery, restaurant, water
source	text	voice, text, photo, receipt, barcode
raw_input	text	Original message/transcript
ai_interpretation	jsonb	Structured AI output
occurred_at	timestampz	Actual event time

Column	Type	Notes
timezone	text	User timezone
inferred_meal_type	text	breakfast/lunch/snack/dinner
confidence_score	numeric	0–1
status	text	processed, needs_review, failed
created_at	timestampz	Auto

7.4 voice_events

Stores voice processing metadata.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
telegram_message_id	text	Source message
audio_temp_path	text	Temporary storage path
raw_transcript	text	ASR output
cleaned_transcript	text	Cleaned output
language	text	en-IN, hi-IN, te-IN, auto
provider_used	text	local_whisper, parakeet, openai, groq
confidence_score	numeric	0–1
processing_time_ms	int	Latency
status	text	success, failed, retry, review
raw_audio_deleted_at	timestampz	Privacy proof
created_at	timestampz	Auto

7.5 food_logs

Structured food entries.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
event_id	uuid	FK user_events
meal_type	text	breakfast/lunch/snack/dinner
foods	jsonb	Extracted foods
contains_sweet	boolean	Pattern detection
contains_fried	boolean	Pattern detection
protein_quality	text	low/medium/high
vegetable_presence	boolean	Yes/no
ai_summary	text	Short user-facing summary
occurred_at	timestampz	Event time

7.6 products

Product intelligence database.

Column	Type	Notes
id	uuid	Primary key
brand	text	Example: Amul
product_name	text	Example: Protein Lassi
normalized_name	text	Search field
category	text	snack, dairy, seeds, drink
barcode	text	Optional
ingredients	text	Ingredient text
nutrition	jsonb	Sugar, sodium, protein, etc.
allergens	jsonb	Optional
processing_level	text	low, medium, high
verification_status	text	unverified, admin_verified
source	text	admin, user, public, api

Column	Type	Notes
created_at	timestampz	Auto
updated_at	timestampz	Auto

7.7 product_logs

Stores product lookup history.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
event_id	uuid	FK user_events
product_id	uuid	FK products
query_text	text	User query
source	text	voice/text/photo/barcode
decision	text	can_have, avoid, occasionally, ask_clarification
reason	text	AI reason
suggested_alternative	text	Optional
confidence_score	numeric	0–1
occurred_at	timestampz	Event time

7.8 reminders

Stores recurring habits.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
habit_name	text	Isabgol, water, walk
reminder_type	text	supplement, water, food, walk, custom
frequency	text	daily, weekly, custom

Column	Type	Notes
schedule_json	jsonb	Flexible schedule
reminder_time	time	Local time
timezone	text	User timezone
active	boolean	Default true
created_from	text	voice/text/admin
created_at	timestampz	Auto

7.9 reminder_events

Stores each reminder occurrence.

Column	Type	Notes
id	uuid	Primary key
reminder_id	uuid	FK reminders
user_id	uuid	FK users
scheduled_at	timestampz	Due time
sent_at	timestampz	Actual sent
user_action	text	taken, later, skip, no_response
action_at	timestampz	User response time
snooze_until	timestampz	If Later
created_at	timestampz	Auto

7.10 behavior_patterns

Stores detected patterns.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
pattern_type	text	sweets, sodium, late_night, low_protein

Column	Type	Notes
period_start	date	Start
period_end	date	End
evidence	jsonb	Supporting events
insight	text	User-facing insight
severity	text	low, medium, high
created_at	timestampz	Auto

7.11 weekly_reports

Stores weekly stories.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
week_start	date	Monday/Sunday rule
week_end	date	End date
food_summary	text	Summary
habit_summary	text	Summary
pattern_summary	text	Patterns
next_week_focus	text	One action
full_report_json	jsonb	Detailed data
sent_at	timestampz	Telegram sent time
created_at	timestampz	Auto

7.12 admin_reviews

Low-confidence human review queue.

Column	Type	Notes
id	uuid	Primary key

Column	Type	Notes
user_id	uuid	Optional
event_id	uuid	Related event
review_type	text	product_match, transcript, food_parse, reminder_parse
original_input	text	User input
ai_output	jsonb	AI result
confidence_score	numeric	0–1
status	text	pending, approved, corrected, rejected
corrected_output	jsonb	Admin correction
reviewed_by	uuid	Admin user
reviewed_at	timestampz	Timestamp
created_at	timestampz	Auto

7.13 consent_settings

Privacy and data sharing.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	FK users
store_transcripts	boolean	Default true
store_raw_audio	boolean	Default false
allow_analytics	boolean	Default false/true depending policy
allow_research	boolean	Default false
allow_partner_sharing	boolean	Default false
created_at	timestampz	Auto
updated_at	timestampz	Auto

8. Authentication

8.1 User Auth

For MVP, Telegram users do not need separate login.

Identification:

telegram_user_id + telegram_chat_id

8.2 Admin Auth

Use **Supabase Auth**.

Supported admin login:

- Email + password
- Google OAuth later

Roles:

Role	Permissions
super_admin	Full access
reviewer	Review low-confidence events
product_admin	Add/edit products
support	View user issues, no sensitive exports
read_only	Dashboard access only

8.3 Row-Level Security

Use RLS on:

- users
- user_profiles
- user_events
- food_logs
- product_logs
- reminders

- weekly_reports
- consent_settings

Admin service role should be used only server-side.

Never expose Supabase service role key to frontend.

9. Hosting and Deployment

9.1 Frontend and API

Vercel

Use for:

- Next.js frontend
 - Admin dashboard
 - Lightweight API routes
 - Telegram webhook if response time is fast
-

9.2 Workers

Use **Railway** / **Fly.io** / **Render** for long-running workers.

Workers needed:

- Voice worker
 - Photo worker
 - Receipt OCR worker
 - Weekly report worker
 - Reminder scheduler worker
 - Pattern detection worker
-

9.3 Database and Storage

Supabase

Use for:

- Postgres database
- File storage

- Auth
- Row-level security
- Admin data

9.4 Local ASR Server

Host separately.

Options:

Option	Use Case
Small CPU server	Low-volume prototype
GPU server	Better speed
Local office machine	Internal prototype
Cloud GPU	Production scaling

10. Third-Party APIs and Services

Service	Purpose	Required in MVP?
Telegram Bot API	Main user interface	Yes
Supabase	DB, auth, storage	Yes
OpenAI / Claude / Gemini	AI reasoning and classification	Yes
OpenWhisper-inspired ASR	Local transcription reference/service	Yes
Whisper.cpp	Local speech-to-text	Yes
Parakeet / sherpa-onnx	Optional fast ASR	Should Have
Groq/OpenAI audio/Deepgram	Cloud fallback transcription	Yes
FFmpeg	Audio conversion	Yes
Sentry	Error monitoring	Yes
Redis/BullMQ	Background jobs	Yes
Resend	Email reports/admin alerts	Later
Stripe	Premium payments	Later
Nutrition/product APIs	Product enrichment	Later

Service	Purpose	Required in MVP?
OCR provider	Receipt extraction	Should Have

11. Key Libraries

11.1 Frontend

```
next
react
typescript
tailwindcss
shadcn/ui
radix-ui
lucide-react
react-hook-form
zod
date-fns
```

11.2 Backend Node

```
@supabase/supabase-js
node-telegram-bot-api or telegraf
zod
bullmq
ioredis
openai
anthropic
ai-sdk
date-fns
uuid
pino
sentry
```

11.3 Python Worker

```
fastapi
uvicorn
```

```
pydantic
python-multipart
ffmpeg-python
requests
openai
numpy
pydub
```

Optional ASR-specific:

```
whisper.cpp binary
sherpa-onnx
faster-whisper
```

12. Environment Variables

12.1 App

```
NEXT_PUBLIC_APP_URL
NODE_ENV
APP_ENV
```

12.2 Supabase

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
SUPABASE_DB_URL
```

12.3 Telegram

```
TELEGRAM_BOT_TOKEN
TELEGRAM_WEBHOOK_SECRET
TELEGRAM_WEBHOOK_URL
```

12.4 AI Providers

```
OPENAI_API_KEY  
ANTHROPIC_API_KEY  
GOOGLE_GENERATIVE_AI_API_KEY  
GROQ_API_KEY
```

12.5 Voice / ASR

```
VOICE_PROVIDER_DEFAULT  
LOCAL_ASR_BASE_URL  
LOCAL_ASR_API_KEY  
OPENAI_TRANSCRIPTION_MODEL  
GROQ_TRANSCRIPTION_MODEL  
DEEPGRAM_API_KEY  
WISPR_FLOW_API_KEY
```

12.6 Storage

```
SUPABASE_AUDIO_BUCKET  
SUPABASE_PHOTO_BUCKET  
SUPABASE_RECEIPT_BUCKET
```

12.7 Queue

```
REDIS_URL  
QUEUE_PREFIX
```

12.8 Monitoring

```
SENTRY_DSN  
LOG_LEVEL
```

12.9 Admin

```
ADMIN_ALLOWED_EMAILS
ADMIN_INVITE_SECRET
```

13. API Routes

13.1 Telegram

```
POST /api/telegram/webhook
```

Purpose:

- Receive Telegram updates
 - Verify secret
 - Route message to input processor
-

13.2 Voice

```
POST /api/voice/process
```

Purpose:

- Create voice processing job
 - Return immediate acknowledgement
-

13.3 Local ASR

```
POST /v1/audio/transcriptions
```

Purpose:

- Accept multipart audio
- Return transcript

Output:

```
{  
  "text": "I ate rice, dal, curd, and one sweet."  
}
```

13.4 Events

```
GET /api/events  
POST /api/events  
PATCH /api/events/:id
```

Purpose:

- Admin/user timeline
- Corrections

13.5 Products

```
GET /api/products/search  
POST /api/products  
PATCH /api/products/:id  
POST /api/products/:id/verify  
POST /api/products/merge
```

Purpose:

- Product graph
- Product correction
- Duplicate merging

13.6 Reminders

```
GET /api/reminders  
POST /api/reminders  
PATCH /api/reminders/:id  
POST /api/reminders/:id/checkin
```

Purpose:

- Create reminders
- Taken/Later/Skip actions

13.7 Weekly Reports

```
POST /api/reports/weekly/generate
GET /api/reports/weekly/:user_id
POST /api/reports/weekly/send
```

14. Background Jobs

Job	Trigger	Purpose
process_voice_note	Telegram voice message	Transcribe and classify
process_photo	Telegram photo	Compress and analyze
process_receipt	Receipt upload	OCR and categorize
send_reminder	Scheduler	Send habit reminder
process_reminder_action	User button click	Taken/Later/Skip
detect_patterns	Daily cron	Detect sweets/sodium/low protein
generate_weekly_report	Weekly cron	Create weekly story
cleanup_raw_audio	Post-transcription	Delete raw audio
admin_review_routing	Low confidence	Send to review queue

15. AI Provider Abstraction

Do not hard-code one AI provider.

Create:

```
/lib/ai/provider-router.ts
```


Provider interface:

```
interface AIProvider {  
  classifyIntent(input: IntentInput): Promise<IntentResult>  
  extractEntities(input: EntityInput): Promise<EntityResult>  
  generateRecommendation(input: RecommendationInput):  
  Promise<RecommendationResult>  
  generateWeeklyReport(input: WeeklyReportInput): Promise<WeeklyReportResult>  
}
```

Supported providers:

- OpenAI
- Claude
- Gemini
- Local model later

16. Voice Provider Abstraction

Create:

```
/lib/voice/provider-router.ts
```

Provider interface:

```
interface VoiceProvider {  
  transcribe(input: VoiceInput): Promise<VoiceTranscript>  
}
```

Response:

```
type VoiceTranscript = {  
  provider: string  
  rawTranscript: string  
  cleanedTranscript: string  
  language?: string  
  confidence?: number  
  processingTimeMs: number  
}
```

Providers:

- local-openwhisper
 - local-whisper
 - parakeet
 - openai
 - groq
 - deepgram
 - wispr-flow
-

17. Folder Structure

```
nowwise/  
  apps/  
    web/  
      app/  
        admin/  
        user/  
        api/  
      components/  
      lib/  
      hooks/  
      styles/  
      public/  
  
  services/  
    telegram-bot/  
      src/  
        webhook.ts  
        message-router.ts  
        reply-service.ts  
  
  voice-service/  
    app/  
      main.py  
      routes/  
        transcriptions.py  
      services/  
        audio_converter.py  
        whisper_adapter.py  
        parakeet_adapter.py  
        cloud_fallback.py  
    utils/  
      cleanup.py
```

```
workers/  
  src/  
    jobs/  
      processVoiceNote.ts  
      processPhoto.ts  
      sendReminder.ts  
      generateWeeklyReport.ts  
      detectPatterns.ts  
  
packages/  
  db/  
    migrations/  
    schema/  
    queries/  
  
  ai/  
    intent-classifier.ts  
    entity-extractor.ts  
    recommendation-engine.ts  
    weekly-report-engine.ts  
    safety-guardrails.ts  
  
  voice/  
    provider-router.ts  
    local-asr-provider.ts  
    cloud-asr-provider.ts  
  
  shared/  
    types/  
    constants/  
    utils/  
  
docs/  
  01-PRD.md  
  02-TRD.md  
  03-App-Flow.md  
  04-UI-UX.md  
  05-Backend-Schema.md  
  06-Implementation-Plan.md
```

18. Naming Conventions

18.1 Database

Use snake_case.

Examples:

```
user_events  
voice_events  
product_logs  
weekly_reports
```

18.2 TypeScript

Use camelCase for variables/functions.

Examples:

```
processVoiceNote()  
createUserEvent()  
generateWeeklyReport()
```

Use PascalCase for types/components.

Examples:

```
VoiceEvent  
ProductDecision  
AdminReviewTable
```

18.3 API Routes

Use kebab-case or resource-based paths.

Examples:

```
/api/products/search  
/api/weekly-reports/generate  
/api/reminders/checkin
```

19. Hard Technical Constraints

19.1 Must-Have Constraints

Constraint	Reason
Must work through Telegram first	Primary distribution
Must support voice notes	Core UX
Must capture timestamp automatically	No manual logging
Must not require photo upload	Poor internet issue
Must delete raw audio by default	Privacy
Must support low-confidence review	AI will make mistakes
Must support fallback ASR	Voice provider failures
Must use queues for heavy jobs	Avoid webhook timeout
Must avoid medical diagnosis claims	Safety
Must avoid selling personal data	Trust

19.2 Performance Constraints

Operation	Target
Telegram webhook acknowledgement	Under 2 seconds
Short voice transcription	Under 3 seconds preferred
Text/product lookup reply	Under 2 seconds
Reminder delivery	Within 1 minute of scheduled time
Weekly report generation	Background, no user wait
Photo processing	Background

19.3 Privacy Constraints

Rule	Requirement
Raw voice audio	Delete after transcription
Raw photo	Store only if needed
Health data	No third-party sharing without opt-in
Admin access	Role-based
Audit logs	Required for sensitive actions
User deletion	Must delete user data

20. MVP Scope

20.1 Build in MVP

Feature	Build?
Telegram bot	Yes
Voice note processing	Yes
Text logging	Yes
Auto timestamp	Yes
Auto meal inference	Yes
Food logging	Yes
Product lookup	Yes
Habit reminder creation	Yes
Isabgol reminder	Yes
Taken/Later/Skip buttons	Yes
Weekly report	Yes
Product database basic	Yes
Admin review queue	Yes
Privacy controls	Yes
Local ASR prototype	Yes

Feature	Build?
Cloud ASR fallback	Yes

20.2 Do Not Build in MVP

Feature	Reason
Full mobile app	Telegram first
Full barcode scanner	V2
Wearables	V2/Future
Insurance analytics	Avoid
Doctor portal	Future
Family mode	Future
Voice biometrics	Avoid
Emotion detection	Avoid
Always-listening assistant	Avoid
Full OpenWhisper desktop integration	Wrong architecture

21. Testing Requirements

21.1 Unit Tests

Test:

- Intent classifier
- Reminder parser
- Product matcher
- Meal type inference
- Voice provider router
- Safety guardrails
- Weekly report generator

21.2 Integration Tests

Test:

- Telegram webhook receives message
 - Voice note downloads correctly
 - Audio conversion works
 - ASR returns transcript
 - Transcript creates event
 - Reminder is scheduled
 - Telegram reply is sent
-

21.3 Voice Accuracy Test Set

Create 500 voice samples.

Sample Type	Count
English food logs	100
Hinglish food logs	100
Telugu/Hindi mixed	50
Product names	100
Reminder commands	50
Restaurant/grocery decisions	50
Noisy supermarket audio	50

22. Monitoring Requirements

Track:

Metric	Why
Telegram webhook failures	Bot reliability
Voice transcription success	Core UX
ASR provider latency	User speed
ASR fallback rate	Provider health

Metric	Why
AI intent accuracy	Product quality
Low-confidence rate	Admin workload
Reminder delivery success	Retention
Weekly report send rate	Engagement
Raw audio deletion success	Privacy compliance
API cost per user	Business viability

23. Security Requirements

Area	Requirement
API secrets	Server-side only
Telegram webhook	Verify secret
Supabase service role	Never expose frontend
Admin dashboard	Auth required
Admin roles	Role-based access
User data	RLS enabled
Raw audio	Temporary only
Audit logs	For admin corrections/deletions
File uploads	Validate type and size

24. Deployment Environments

Use three environments:

Environment	Purpose
local	Developer testing
staging	Internal QA
production	Real users

Each environment needs separate:

- Supabase project or schema
 - Telegram bot token
 - Storage buckets
 - AI keys
 - Redis queue
 - Webhook URL
-

25. Claude/Codex Technical Instructions

When implementing NowWise:

1. Do not choose another stack.
 2. Use Next.js + TypeScript for web/admin.
 3. Use Supabase Postgres for database.
 4. Use Telegram Bot API as primary channel.
 5. Use Python/FastAPI for voice service.
 6. Use FFmpeg for audio conversion.
 7. Use local OpenWhisper-inspired ASR first, with cloud fallback.
 8. Use queues for heavy jobs.
 9. Store every interaction as a user_event.
 10. Do not ask users to manually select date/time.
 11. Do not make medical diagnosis claims.
 12. Do not sell/share personal data.
 13. Delete raw audio after transcription.
 14. Route low-confidence AI outputs to admin review.
 15. Keep voice provider and AI provider modular.
-

26. Final TRD Decision

The correct technical architecture for NowWise is:

```
Telegram-first frontend
+ Next.js admin dashboard
+ Supabase Postgres
+ Python voice service
+ OpenWhisper-inspired local ASR
+ Cloud fallback
+ AI intent/recommendation engine
+ Background workers
```

- + Admin review queue
- + Privacy-first data controls

Do not build a mobile app first.

Do not build a photo scanner first.

Do not embed full OpenWhisper desktop app into the backend.

Build the core system around this:

User speaks to Telegram. NowWise understands, logs, remembers, recommends, reminds, and reports.